# GMC LANKA CREATIONS

**By**

**KEERTHI GOPALAVANNAN**

**SE/2021/020**

**ASWINI PARAMESWARAN**

**SE/2021/023**

**BALASUBRAMANIYAM THUSHAKARAN**

**SE/2021/052**

**THARSHANA SARATHCHANDAR**

**SE/2021/056**

**A report submitted in partial fulfillment of the requirements for the degree of Bachelor of Science Honors in Software Engineering (B. Sc.SE)**

**Software Engineering Teaching Unit**

**Faculty of Science**

**University of Kelaniya**

**Sri Lanka**

**2026**

**Version history**

| Version No. | Date | Author | Summary of Changes |
|---|---|---|---|
| 01 | 2026/03/23 | Team | Initial draft of SDS document |
| 02 | 2026/03/29 | Team | Updated diagram |
| 03 | 2026/04/09 | Team | Added UI/UX design |
| 04 | 2026/04/17 | Team | Final review and corrections |

## Table of Content

Version history ..... ii
Table of Content ..... iii
List of Figures ..... vi
CHAPTER 1 ARCHITECTURAL DESIGN ..... 1
1.1 High-Level Architecture ..... 1
1.1.1 Architecture Overview ..... 1
1.1.2 Architecture Style ..... 2
1.2 Cloud-Native Architecture (AWS-Based) ..... 2
1.2.1 Key AWS Components ..... 2
1.3 Justification for Architecture Choice ..... 3
1.3.1 Simplicity ..... 3
1.3.2 Separation of concerns ..... 4
1.3.3 Cost-effectiveness ..... 4
1.3.4 Flexibility ..... 4
1.3.5 Industry relevance ..... 4
1.4 Scalability and Maintainability Considerations ..... 4
1.4.1 Scalability ..... 4
1.4.2 Maintainability ..... 5
1.4.3 Additional considerations ..... 5
CHAPTER 2 DATABASE DESIGN ..... 6
2.1 ER Diagram ..... 6
2.2 Normalization explanation ..... 7
2.2.1 What is Normalization? ..... 7
2.2.2 First Normal Form (1NF) ..... 7
2.2.3 Second Normal Form (2NF) ..... 7
2.2.4 Third Normal Form (3NF) ..... 8

2.2.5 BCNF (Advanced Rule) ..... 9
2.2.6 Fourth Normal Form (4NF) ..... 9
CHAPTER 3 CLASS DIAGRAM ..... 10
CHAPTER 4 SEQUENCE DIAGRAMS ..... 11
4.1 User Login ..... 11
4.2 User Register ..... 12
4.3 User Place Order ..... 13
4.4 User Booking Appointment ..... 14
4.5 User Submit Custom Design ..... 15
4.6 Admin Manage Products ..... 16
4.7 Admin add User ..... 17
4.8 Technician Manage Maintenance ..... 18
CHAPTER 5 UI/UX DESIGN ..... 19
5.1 Wireframes for key user interfaces ..... 19
5.2 User flow / Navigation Diagrams ..... 30
5.2.1 Customer Flow ..... 30
5.2.2 Admin Flow ..... 31
5.2.3 Technician Flow ..... 32
5.3 Accessibility Considerations (WCAG 2.1 Compliance) ..... 32
CHAPTER 6 SECURITY DESIGN ..... 34
6.1 Authentication Mechanism ..... 34
6.2 Authorization Levels ..... 34
6.3 Data Protection Approach ..... 35
6.4 Input Validation And Sanitization Strategy ..... 36
6.5 Encryption Standards For Data at Rest and in Transit ..... 37
CHAPTER 7 DEPLOYMENT DESIGN ..... 38
7.1 Hosting Plan ..... 38
7.2 Environment Specification ..... 38
7.3 Deployment Strategy ..... 39
7.4 Network and Security Configuration ..... 39

7.5 Network Diagram.....40
CHAPTER 8 REFERENCES.....41

**List of Figures**

Figure 2.1 Entity Relationship (ER) Diagram ..... 06
Figure 3.1 Class Diagram ..... 10
Figure 4.1 Sequence Diagram of User Login ..... 11
Figure 4.2 Sequence Diagram of User Register ..... 12
Figure 4.3 Sequence Diagram of User Place Order ..... 13
Figure 4.4 Sequence Diagram of User Book Appointment ..... 14
Figure 4.5 Sequence Diagram of User Submit Custom Design ..... 15
Figure 4.6 Sequence Diagram of Admin Manage Products ..... 16
Figure 4.7 Sequence Diagram of Admin Add User ..... 17
Figure 4.8 Sequence Diagram of Technician Manage Maintenance ..... 18
Figure 5.1 Wireframe of User Register ..... 19
Figure 5.2 Wireframe of User Login ..... 20
Figure 5.3 Wireframe of Product Catalogue ..... 21
Figure 5.4 Wireframe of Product Details ..... 22
Figure 5.5 Wireframe of Product List Page ..... 23
Figure 5.6 Wireframe of Checkout Page ..... 24
Figure 5.7 Wireframe of Payment Page ..... 25
Figure 5.8 Wireframe of Admin Overview Page ..... 26
Figure 5.9 Wireframe of Admin Manage Orders ..... 27
Figure 5.10 Wireframe of Admin Manage Appointments ..... 28
Figure 5.11 Wireframe of Admin Manage Users ..... 29
Figure 5.12 User Flow Diagram of Customer ..... 30
Figure 5.13 User Flow Diagram of Administrator ..... 31
Figure 5.14 User Flow Diagram of Technician ..... 32
Figure 7.1 Network Diagram ..... 40

# CHAPTER 1 ARCHITECTURAL DESIGN

## 1.1 High-Level Architecture

A layered architecture (N-tier architecture) combined with the Model-View-Controller (MVC) design pattern is adopted by the system. The implementation is carried out using the Spring Boot framework.

### 1.1.1 Architecture Overview

The system is structured into multiple layers to ensure separation of concerns and maintainability. The layers are described as follows

- Presentation layer (client layer)

The user interface is provided through the presentation layer. User interactions are handled and forwarded to the backend services via RESTful APIs. Technologies such as React JS, HTML, CSS, and JavaScript are utilized.

- Controller layer (MVC – controller)

HTTP requests are received and processed within the controller layer. Requests are mapped to appropriate service components. This layer is implemented using Spring Boot REST controllers.

- Service layer (business logic layer)

Core business logic is implemented within the service layer. Data processing and enforcement of system rules are carried out. Communication between the controller layer and repository layer is facilitated.

- Repository layer (data access layer)

Database operations are managed within the repository layer. CRUD operations are performed using Spring Data JPA and Hibernate frameworks.

- Database layer

Application data is stored within the database layer. Relational database management systems such as MySQL is used.

1.1.2 Architecture Style

The architectural design is based on the following

- Primary pattern: Layered architecture
- Design pattern: MVC pattern
- Communication: RESTful APIs using JSON format

Communication between the frontend and backend is performed using RESTful APIs over HTTPS.

**1.2 Cloud-Native Architecture (AWS-Based)**

The system is designed using cloud-native principles and is deployed on Amazon Web Services (AWS) using virtual machine-based infrastructure.

1.2.1 Key AWS Components

- Computer layer

The application is deployed on AWS EC2 instances. The Spring Boot application is executed as a standalone .jar file. Full control over the server environment is provided.

- Database layer

AWS RDS is utilized for database management. MySQL databases are supported. Automated backups, scalability, and high availability are ensured.

- Storage layer

AWS S3 is used for storing static resources such as images, documents, and optional log files.

- Frontend hosting

The frontend application is hosted using AWS S3 and may be delivered via AWS CloudFront for efficient content distribution.

- Networking and security

Network isolation is achieved using AWS VPC (Virtual Private Cloud). Traffic is controlled through security groups. Access control is managed using AWS IAM roles and policies.

- Deployment and management

The system is deployed either manually or through CI/CD pipelines. Tools such as AWS CodePipeline or GitHub Actions may be utilized.

- Monitoring and logging

System performance and logs are monitored using AWS CloudWatch. Alerts and notifications are configured for system reliability.

**1.3 Justification for Architecture Choice**

The selected architecture is justified based on the following factors

1.3.1 Simplicity

Development, deployment, and management are simplified through the use of Spring Boot and AWS EC2. It is suitable for academic and medium-scale applications.

1.3.2 Separation of concerns

System components are divided into distinct layers. This improves code organization and readability.

1.3.3 Cost-effectiveness

AWS services such as EC2 and RDS (Relational Database Service) provide a cost-efficient solution for deployment and database management.

1.3.4 Flexibility

The system may be extended or modified with minimal impact on other components. Future migration to microservices or container-based architecture is supported.

1.3.5 Industry relevance

Widely used technologies and cloud platforms are adopted. This aligns the system with modern software engineering practices.

**1.4 Scalability and Maintainability Considerations**

1.4.1 Scalability

System scalability is ensured through the following approaches

- Vertical scaling is achieved by increasing EC2 instance capacity.
- Horizontal scaling can be implemented by deploying multiple EC2 instances with load balancing (AWS ELB).
- Database scalability is supported through RDS read replicas.
- Backend performance is improved through query optimization and efficient resource usage.

### 1.4.2 Maintainability

System maintainability is ensured through the following

- A layered structure is maintained for modular development.
- Code reusability is achieved through centralized business logic in the service layer.
- A standard framework (Spring Boot) is utilized to ensure consistency.
- Testing is supported through unit testing (JUnit) and integration testing (Spring Boot Test).
- Monitoring and logging are handled using AWS CloudWatch for quick issue resolution.

### 1.4.3 Additional considerations

- Security is implemented using Spring Security and AWS IAM roles.
- Error handling is centralized using “@ControllerAdvice”.
- API versioning is applied to maintain backward compatibility.

Layered MVC architecture has been adopted using Spring Boot and deployed on AWS cloud infrastructure (EC2, RDS, S3) without containerization. This architectural approach ensures simplicity, scalability, maintainability, and alignment with industry practices.

# CHAPTER 2 DATABASE DESIGN

## 2.1 ER Diagram

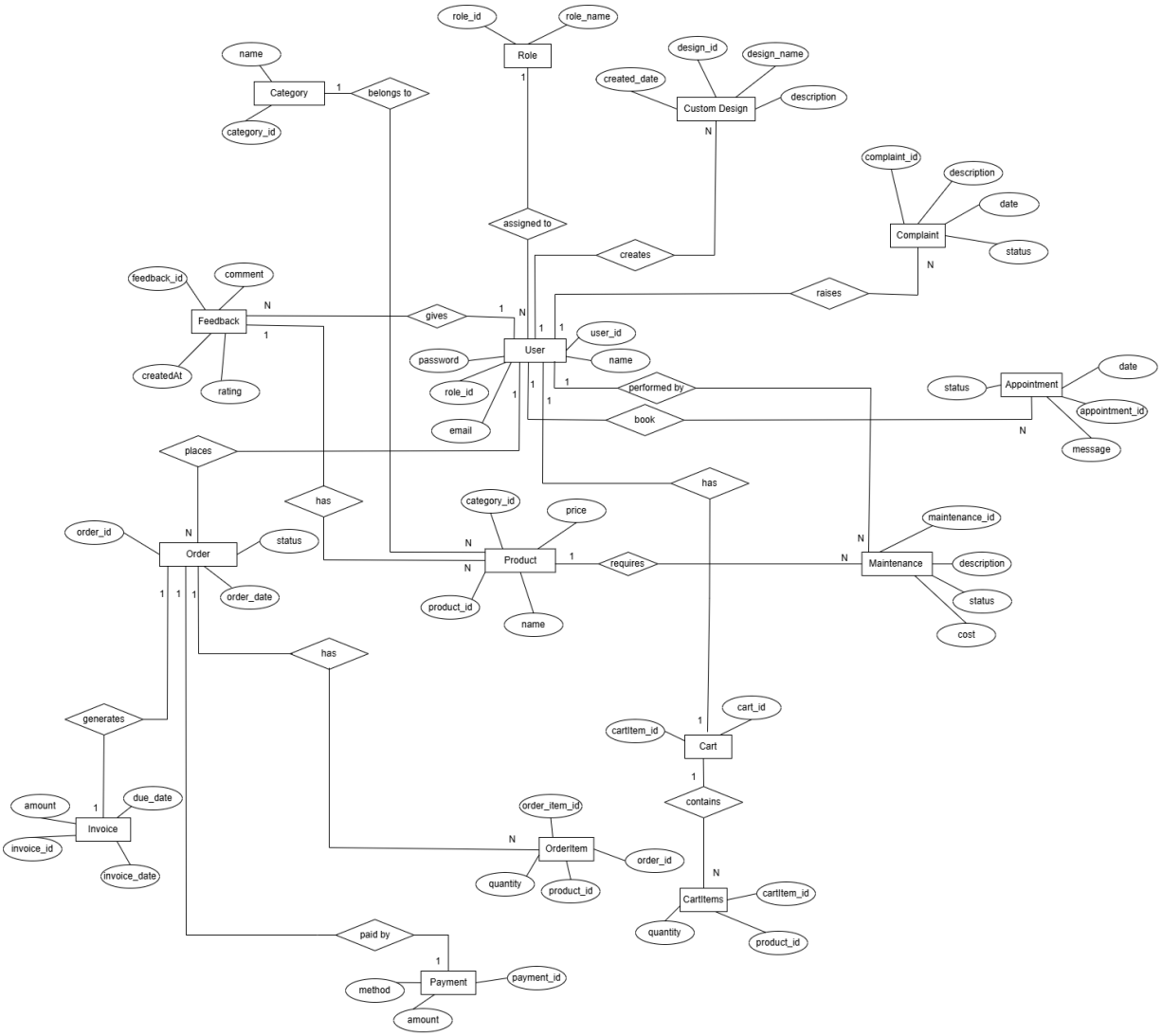



*Figure 2.1 ER Diagram*

## 2.2 Normalization explanation

### 2.2.1 What is Normalization?

Normalization means organizing the database in a proper and structured way so that the data is stored efficiently. It helps to remove duplicate (repeated) data and ensures that the information remains accurate and consistent. By doing this, normalization makes the database easier to manage, update, and maintain without causing errors or confusion

### 2.2.2 First Normal Form (1NF)

Rule

- Each column should have only one value (no multiple values)
- No repeating data in the same field
- Every table must have a primary key

In our system,

- All tables like User, Product, Order, Complaint, etc. have unique IDs
- All values are single (not multiple)

So, our database satisfies 1NF

### 2.2.3 Second Normal Form (2NF)

Rule

- Must already be in 1NF
- All data should depend on the whole primary key

In our system,

- Each table uses a single primary key (like product_id, user_id)
- So no partial dependency problem

Example: -

- In OrderItem, quantity depends fully on order_item_id

So, our database satisfies 2NF

### 2.2.4 Third Normal Form (3NF)

Rule

- Must be in 2NF
- No indirect dependency (no “middle” dependency)

In our system,

- User → Role
    - user stores only role_id
    - role name is in Role table
- Product → Category
    - product stores only category_id
    - category name is in Category table

This avoids duplication

- In Payment, amount should not depend on other tables indirectly

So, our database satisfies 3NF

2.2.5 BCNF (Advanced Rule)

Rule

- Every dependency should come from a key

In our system,

- email in User should be unique

If you add UNIQUE constraint: email UNIQUE

Then our database satisfies BCNF

2.2.6 Fourth Normal Form (4NF)

Rule

- No multiple independent relationships in one table

In our system,

- Many-to-many relationships are separated properly:
    - OrderItem
    - CartItems

Example

- A product in orders and cart is stored in separate tables

So, our database satisfies 4NF

# CHAPTER 3 CLASS DIAGRAM

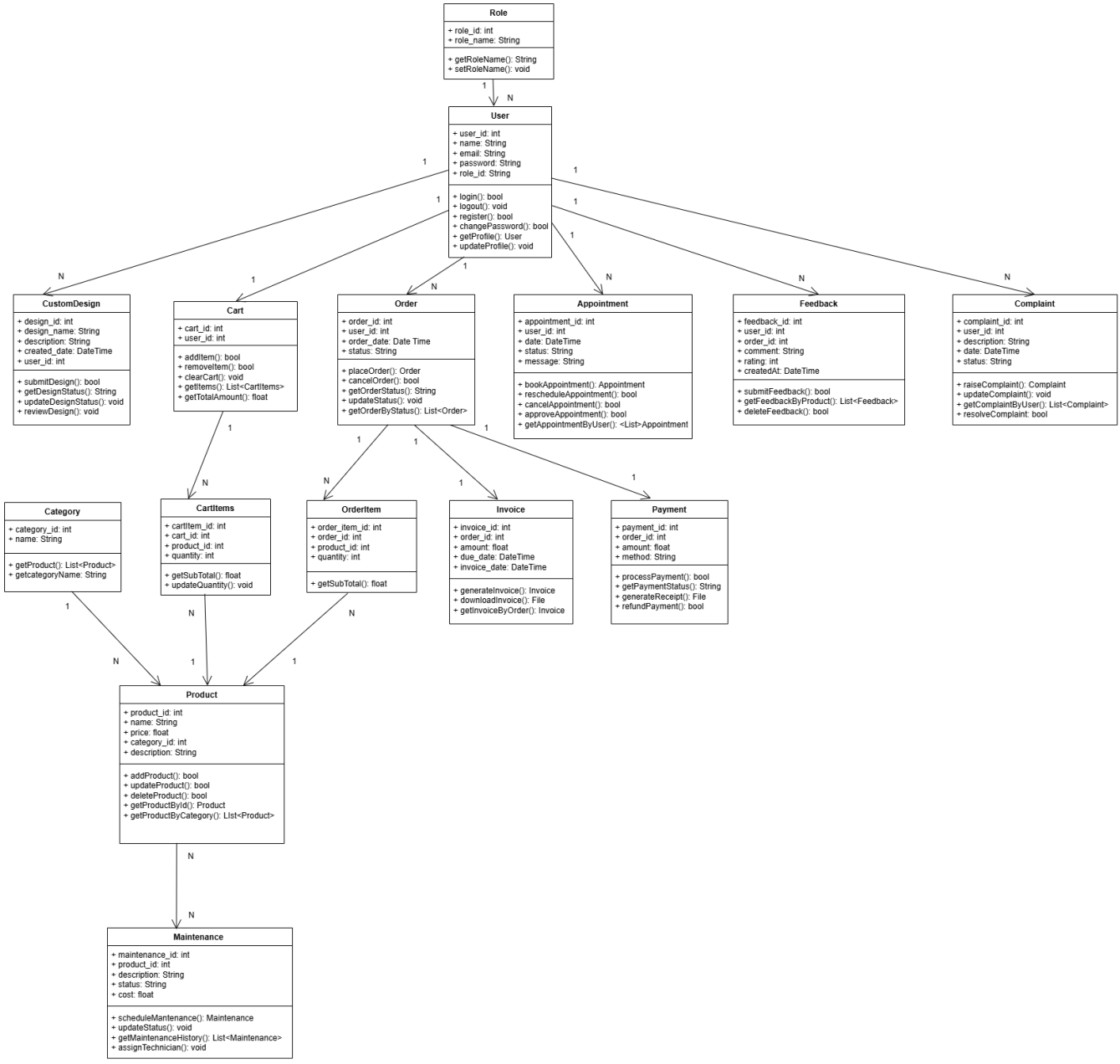



*Figure 3.1 Class Diagram*

# CHAPTER 4 SEQUENCE DIAGRAMS

## 4.1 User Login

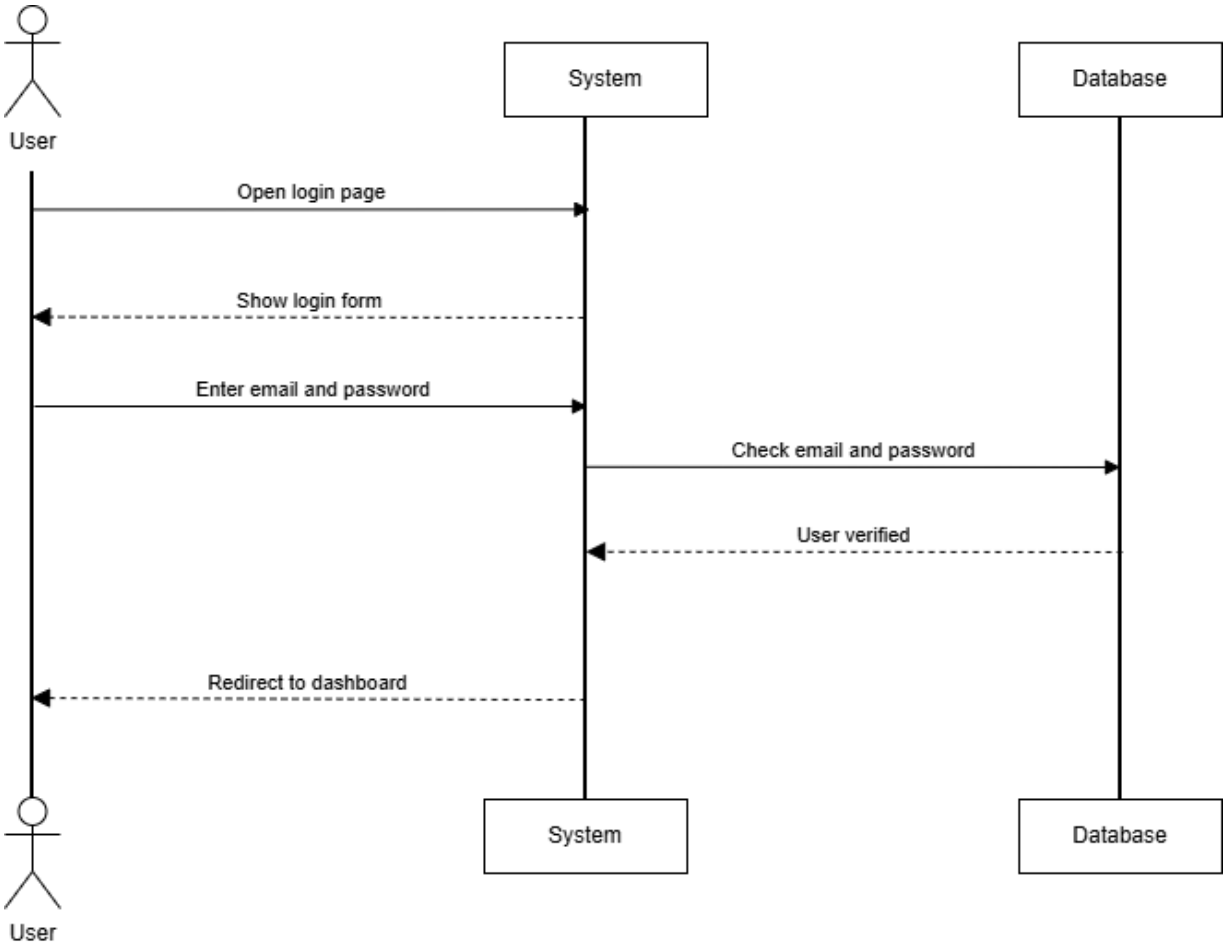



*Figure 4.1 Sequence diagram of User Login*

### 4.2 User Register

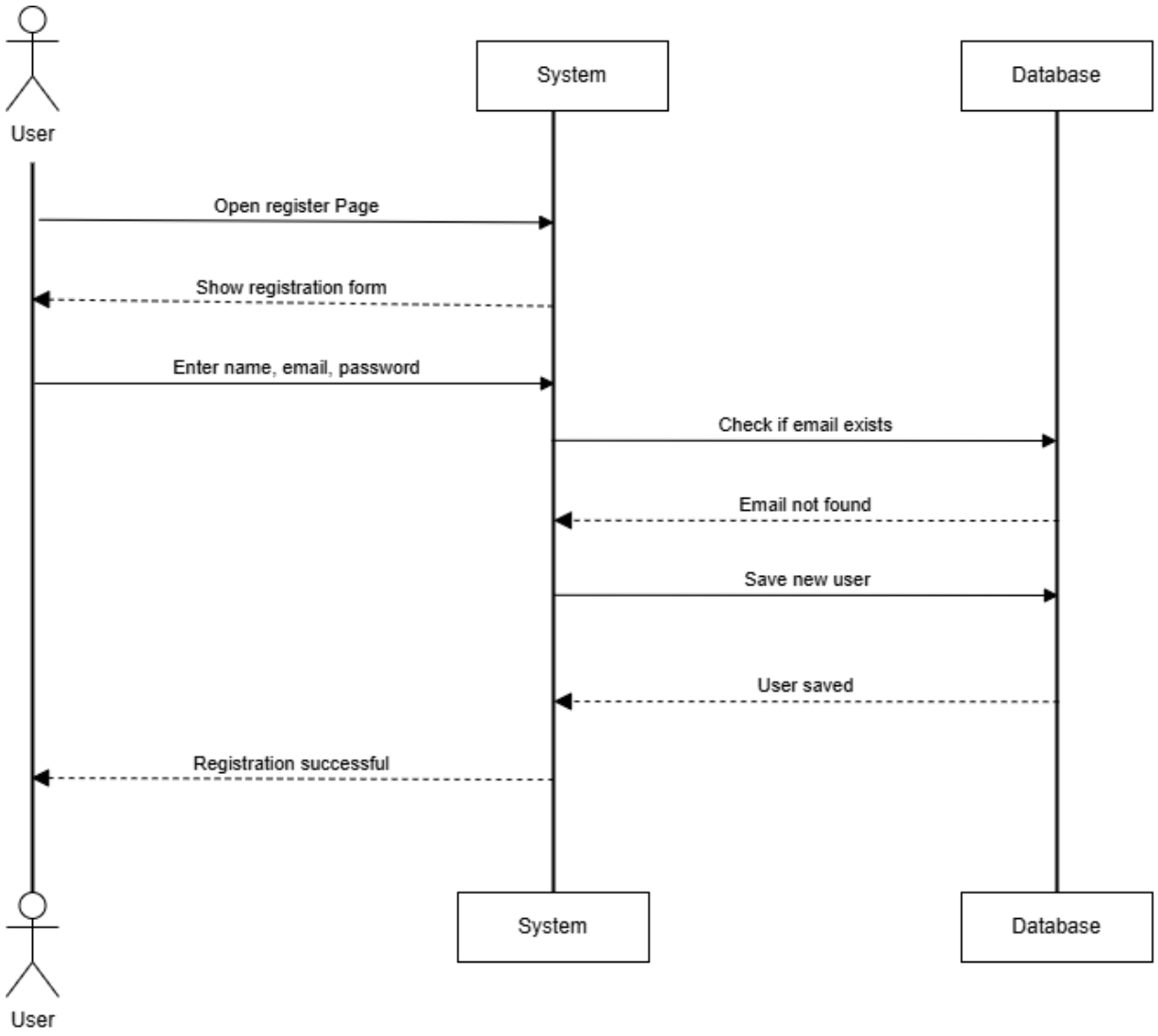



*Figure 4.2 Sequence diagram of User Register*

## 4.3 User Place Order

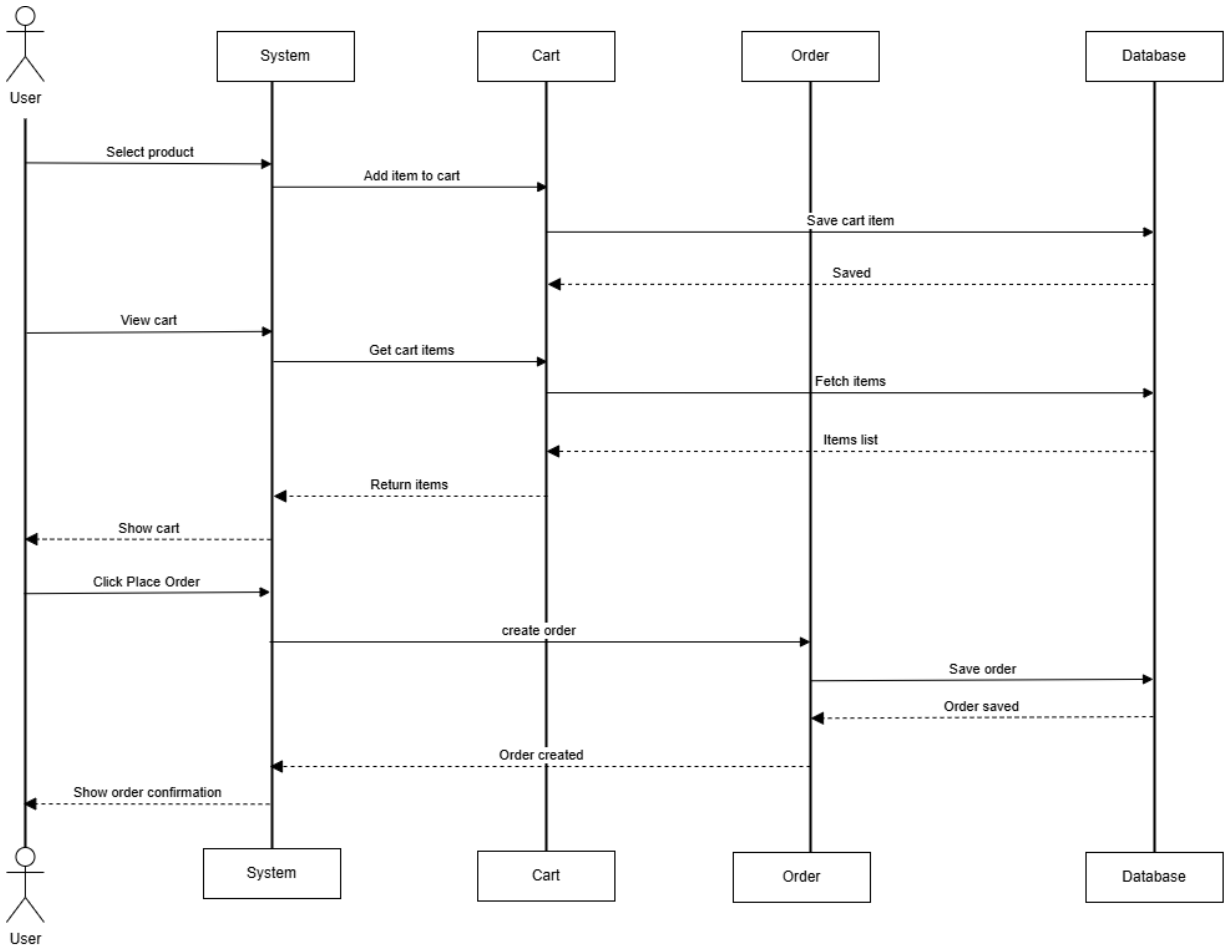



*Figure 4.3 Sequence diagram of User Place Order*

### 4.4 User Booking Appointment

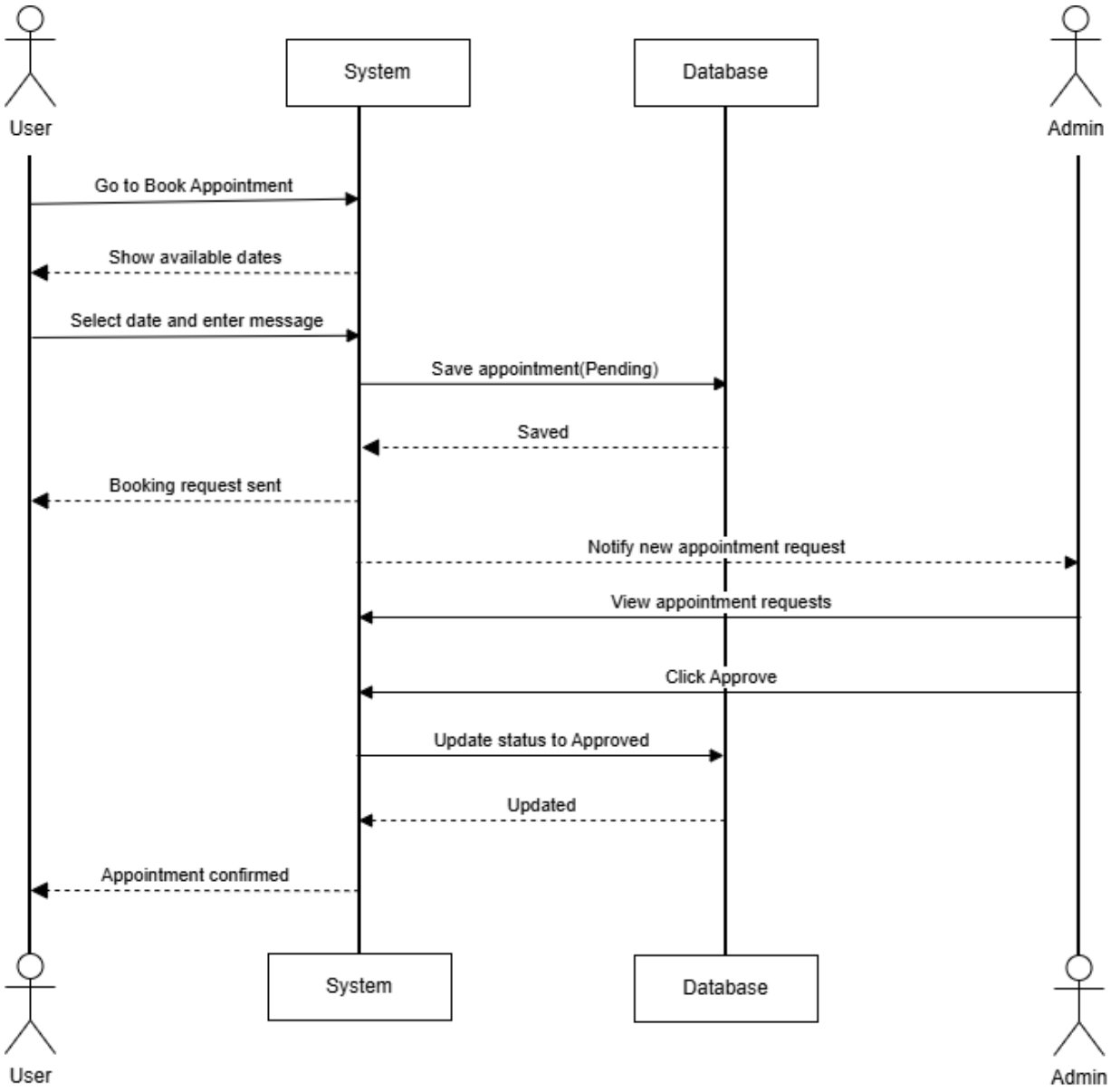



*Figure 4.4 Sequence diagram of User Booking Appointment*

### 4.5 User Submit Custom Design

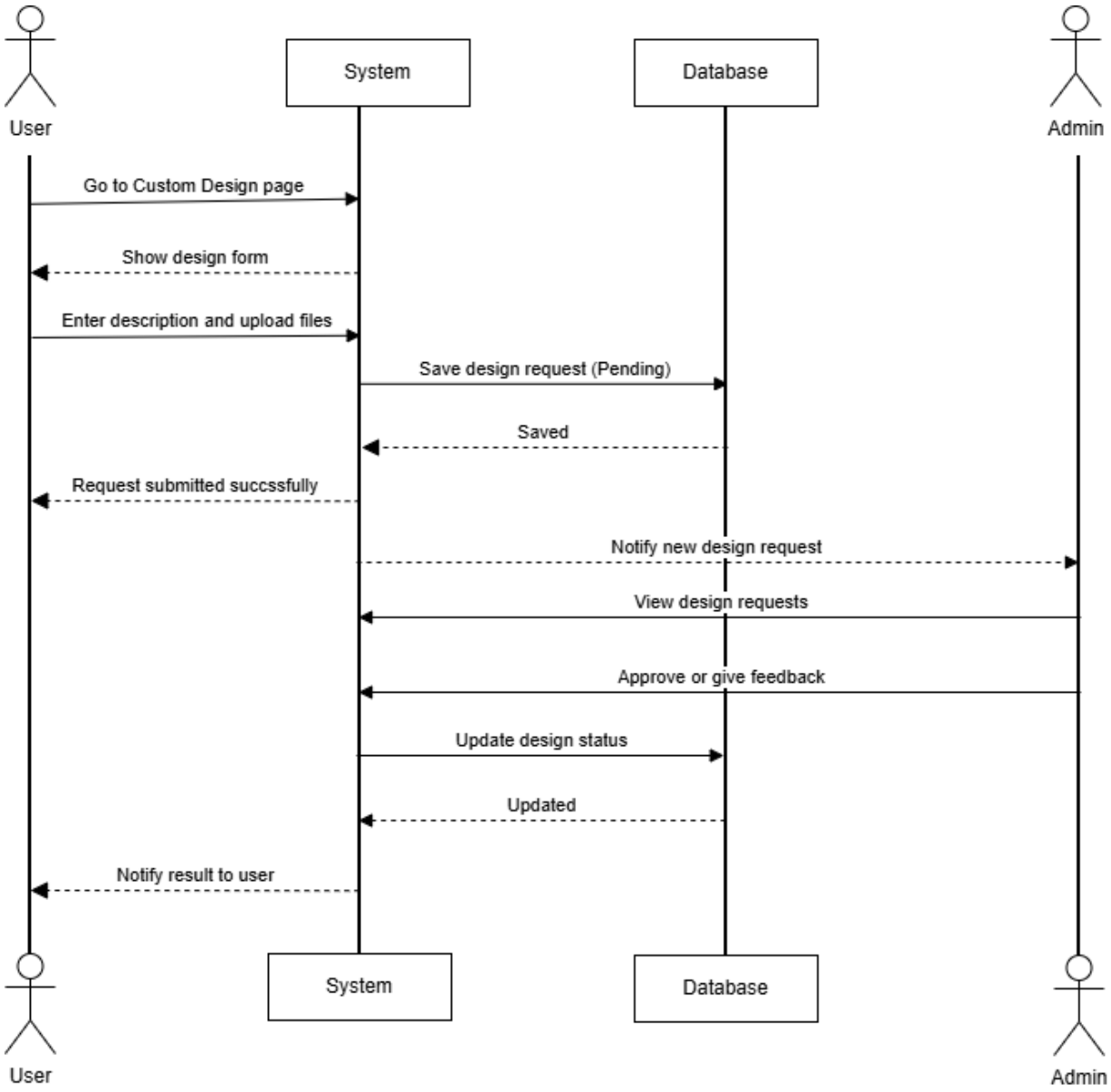



*Figure 4.5 Sequence diagram of User Submit Custom Design*

### 4.6 Admin Manage Products

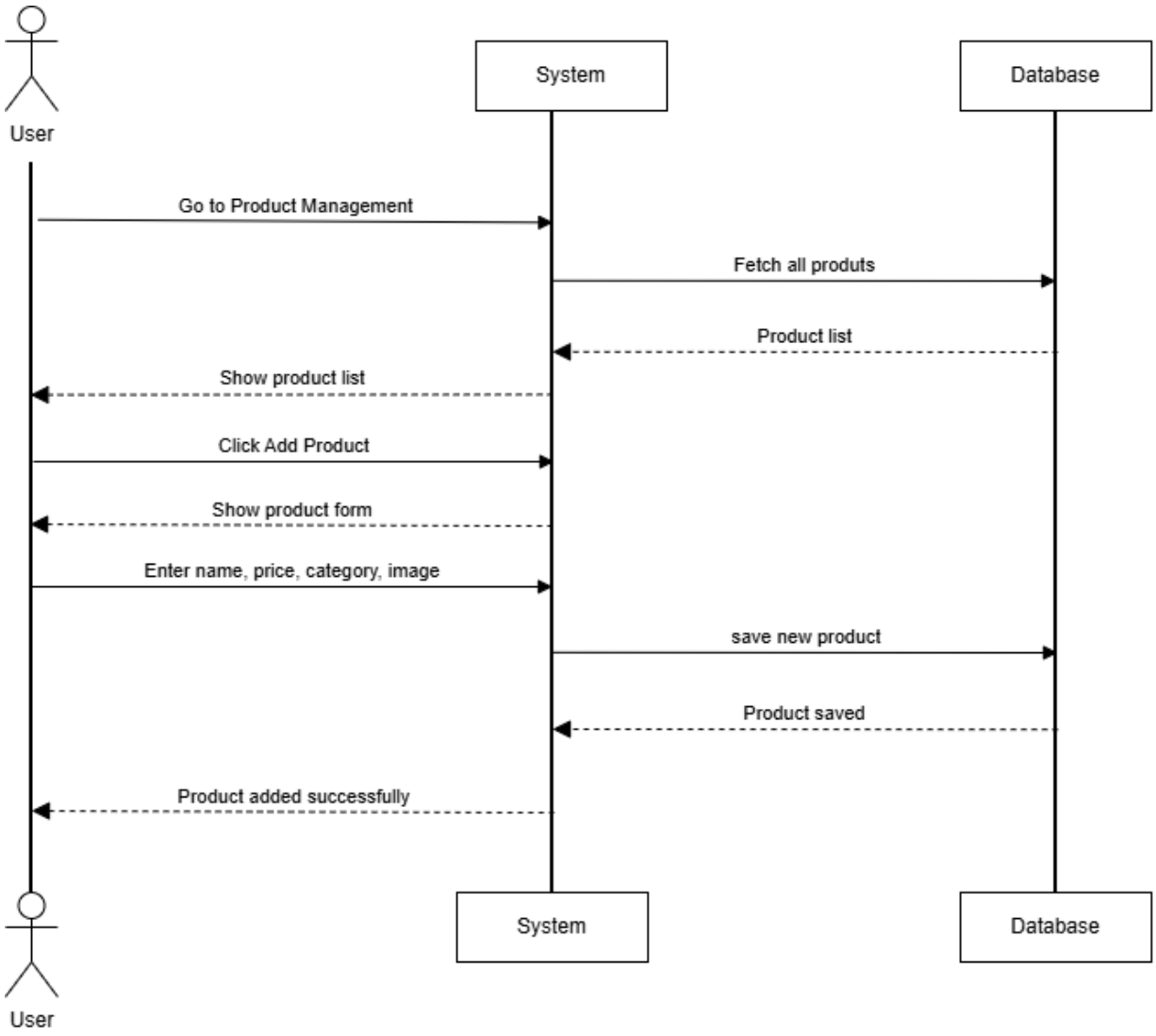



*Figure 4.6 Sequence diagram of Admin Manage Products*

### 4.7 Admin add User

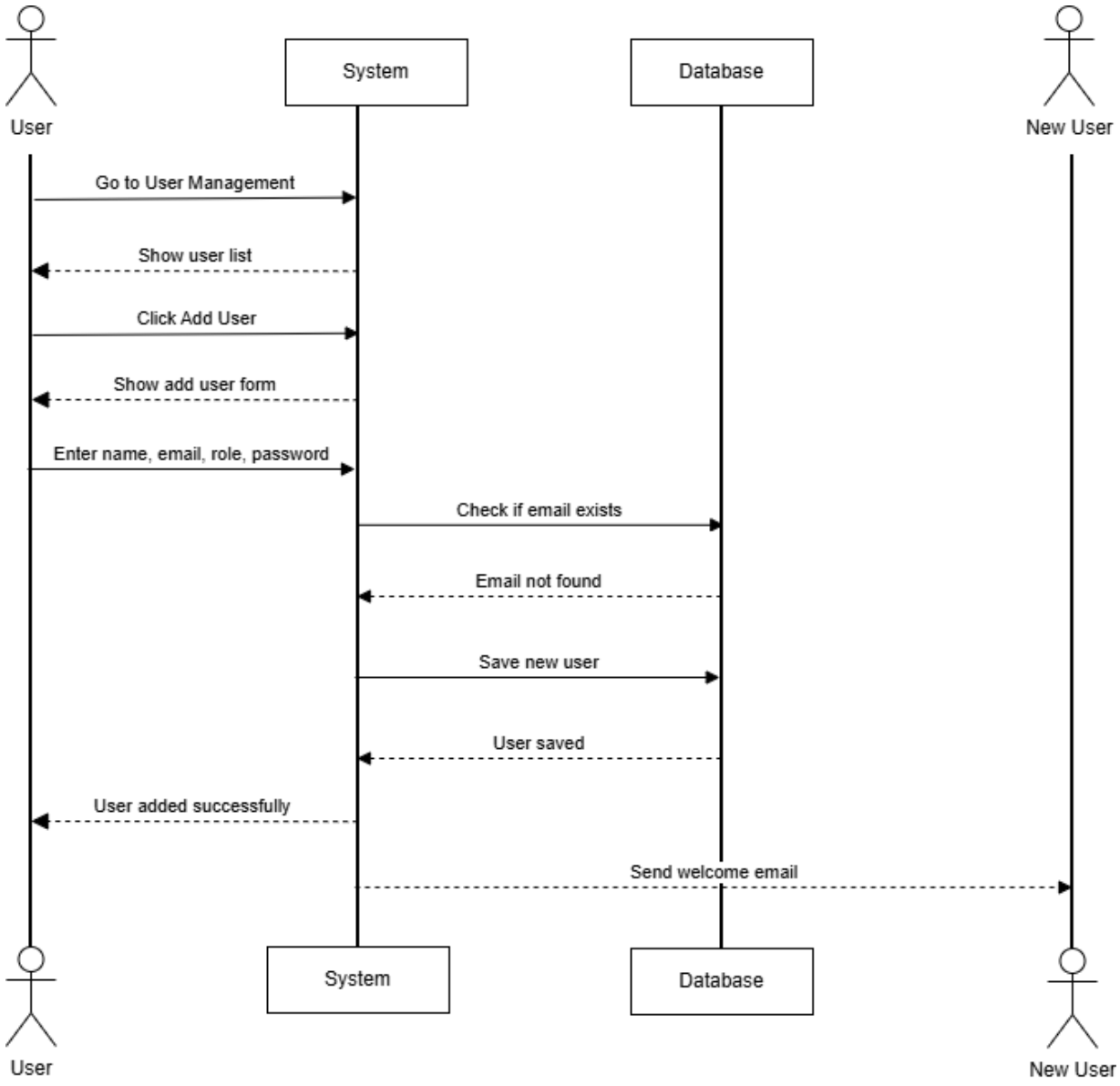



*Figure 4.7 Sequence diagram of Admin Add User*

### 4.8 Technician Manage Maintenance

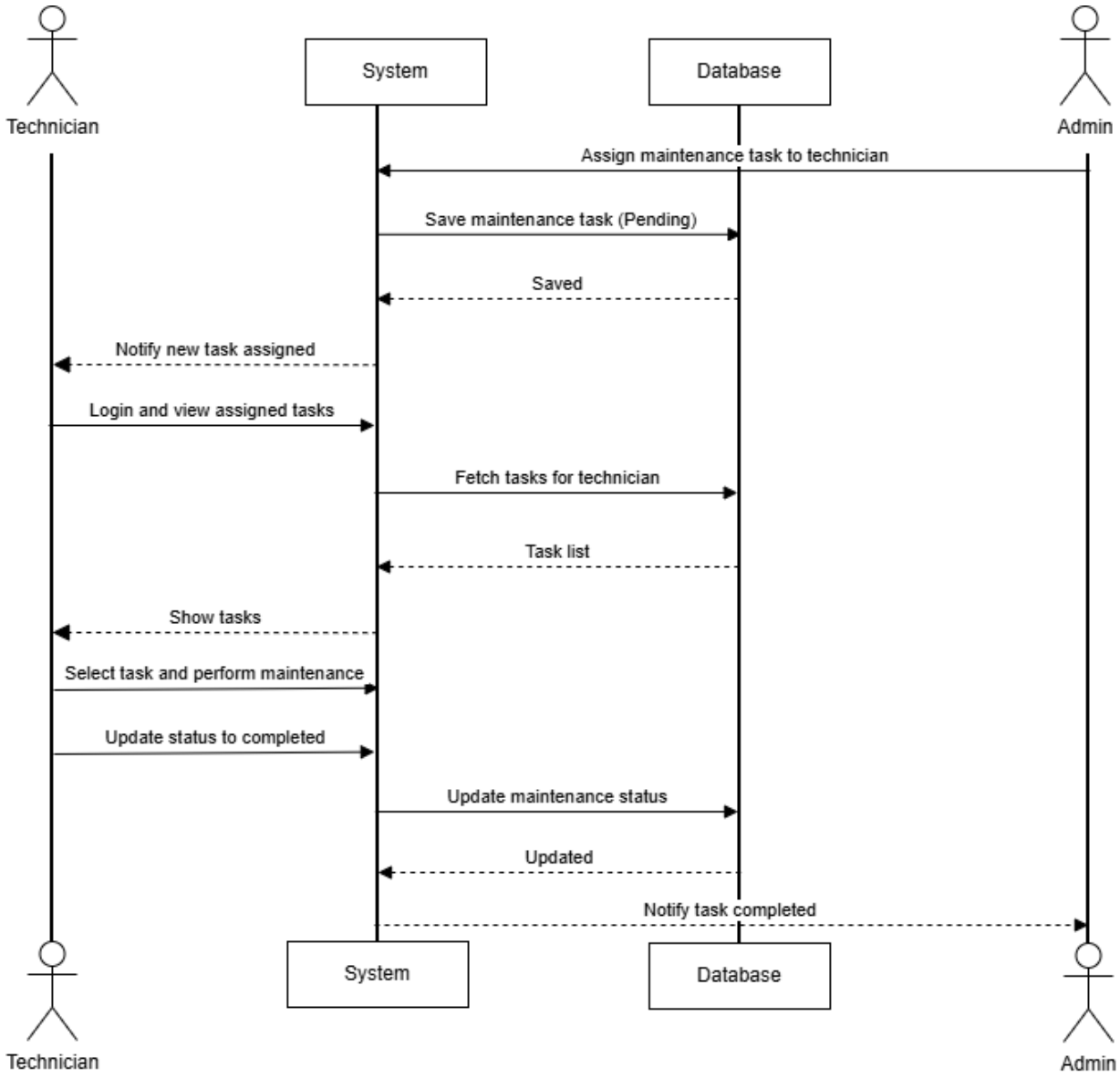



*Figure 4.8 Sequence diagram of Technician Manage Maintenance*

# CHAPTER 5 UI/UX DESIGN

## 5.1 Wireframes for key user interfaces

User Register

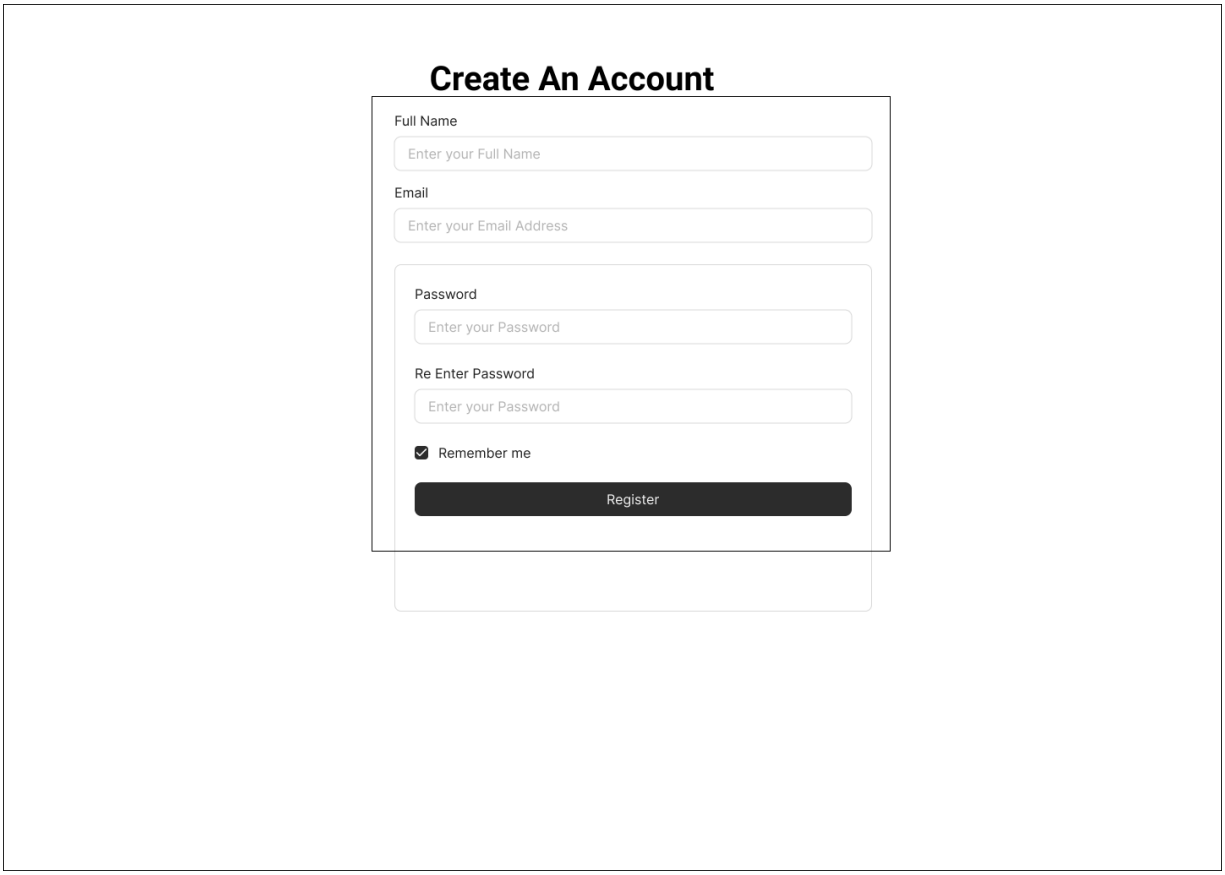



*Figure 5.1 Wireframe of User Register*

User Login

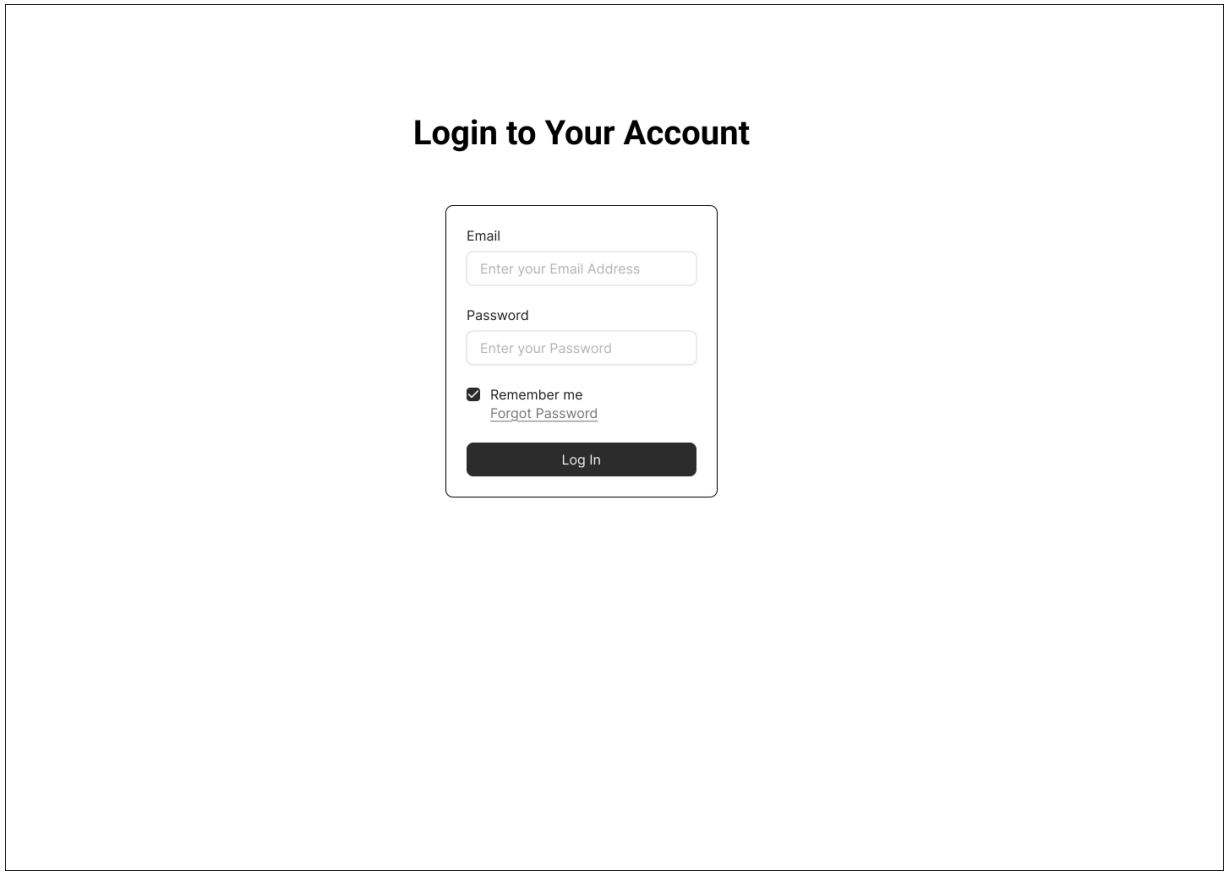



*Figure 5.2Wireframe of User Login*

Product Catalog

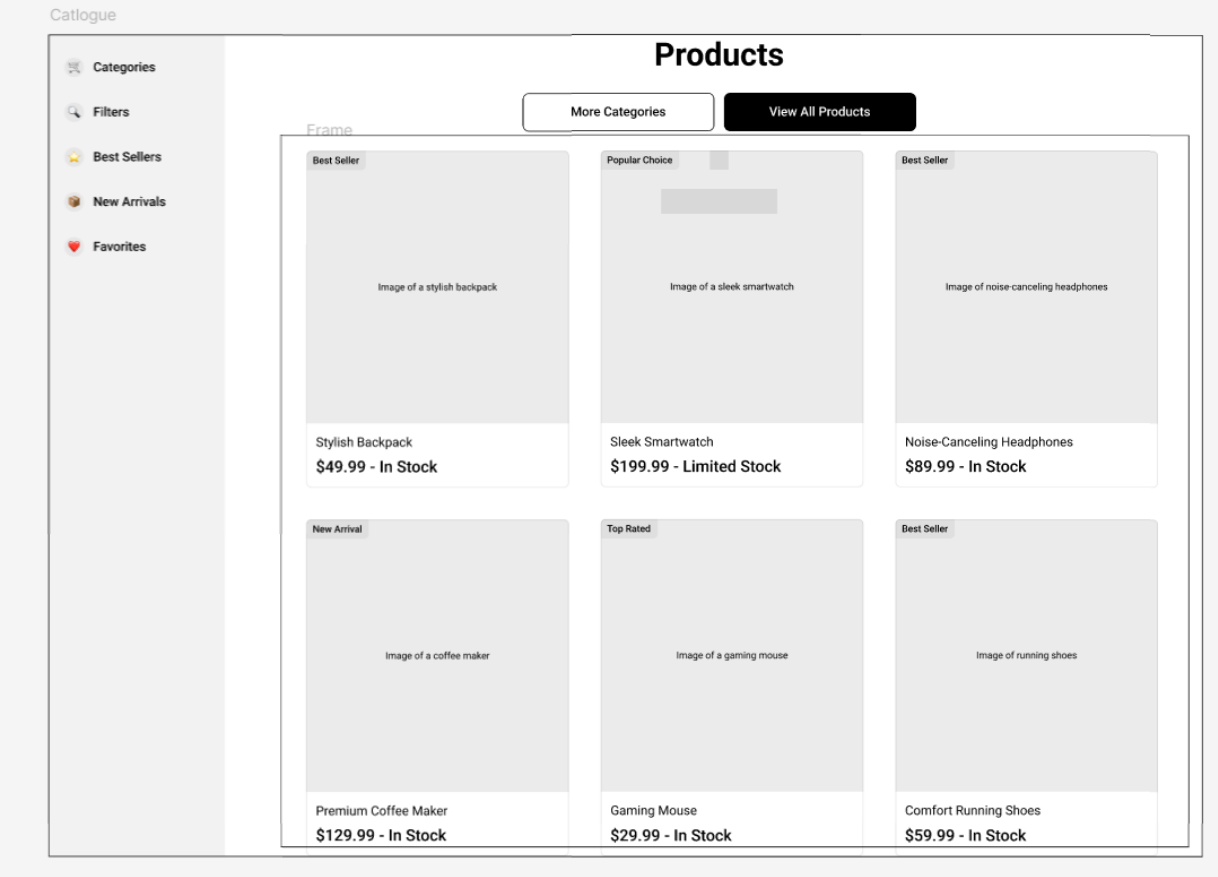



*Figure 5.3 Wireframe of Product Catalog*

Product Details

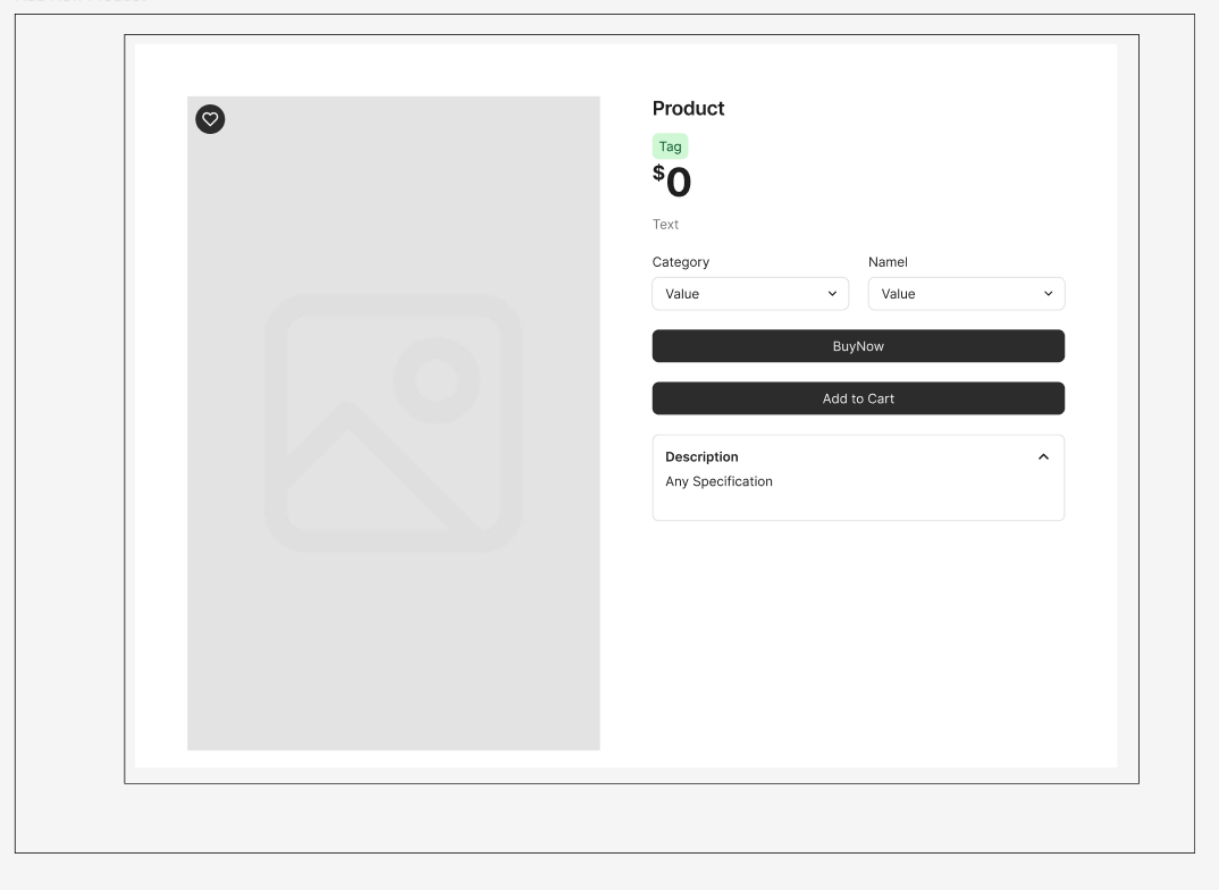



*Figure 5.4 Wireframe of Product Details*

Product List Page

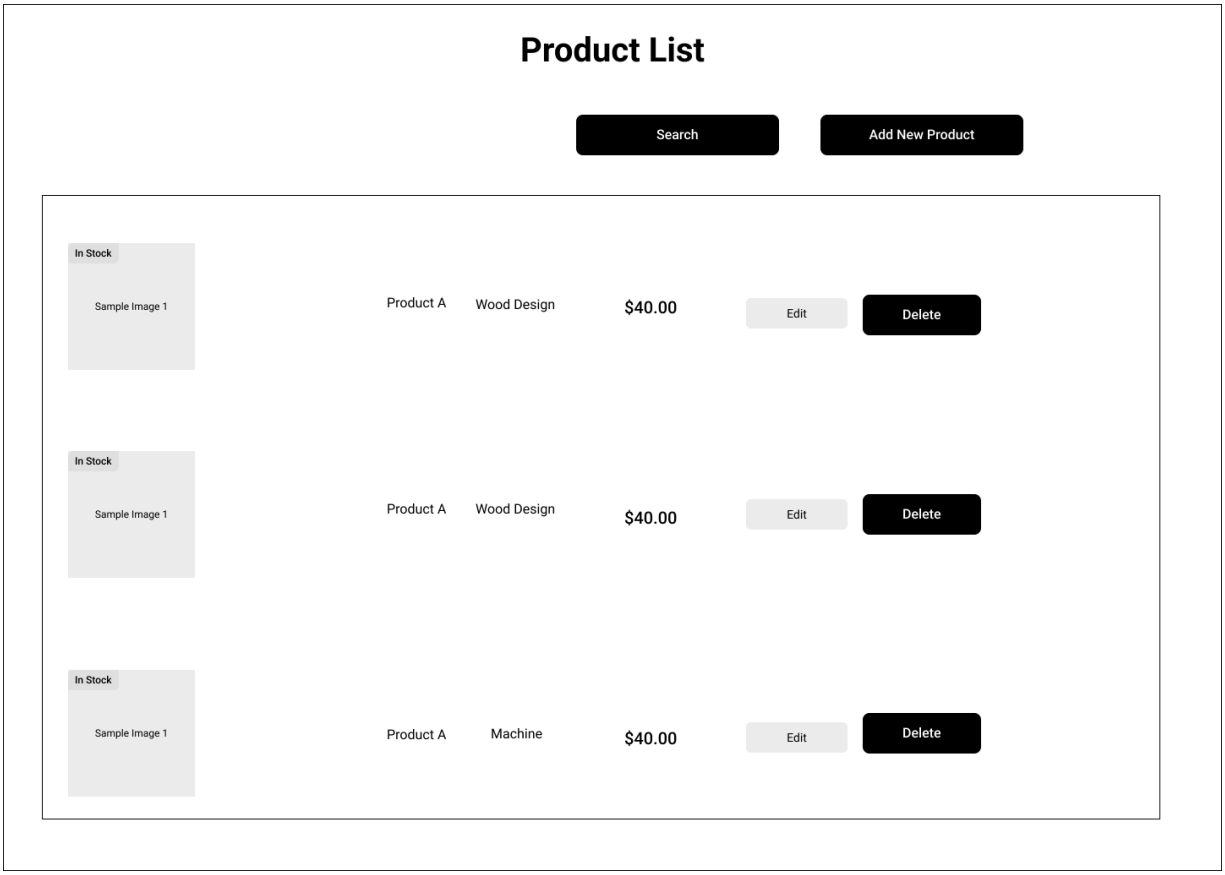



*Figure 5.5 Wireframe of Product List*

Checkout Page

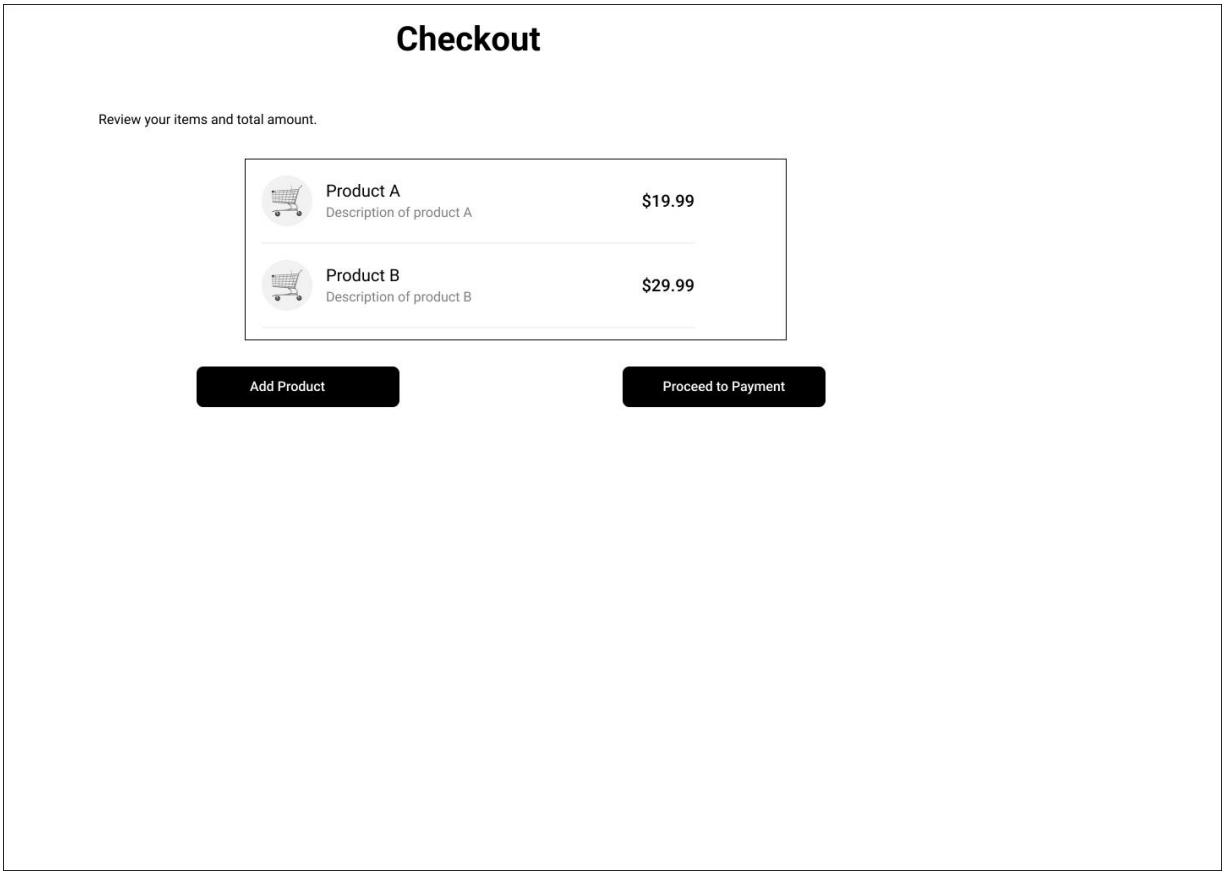



*Figure 5.6 Wireframe of Checkout Page*

Payment Page

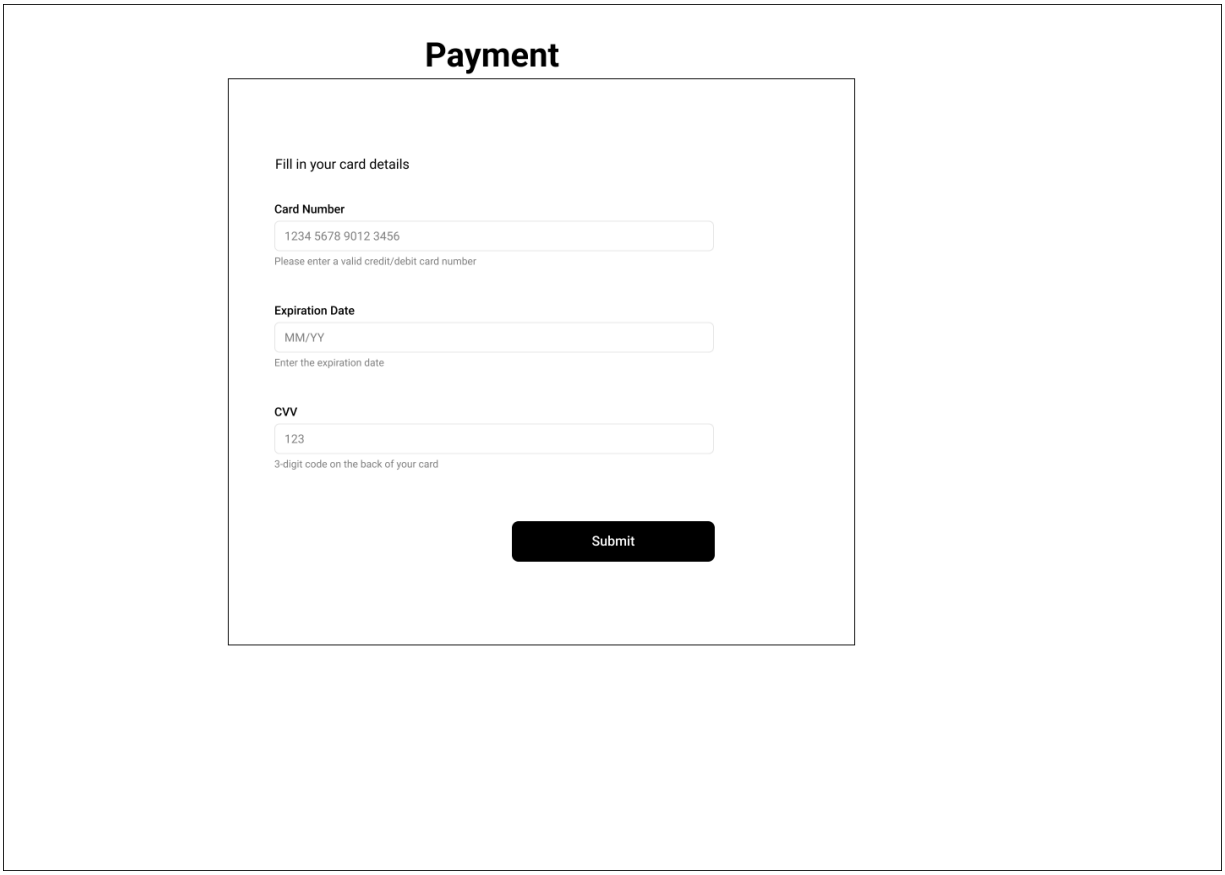



*Figure 5.7 Wireframe of Payment Page*

Admin Overview Page

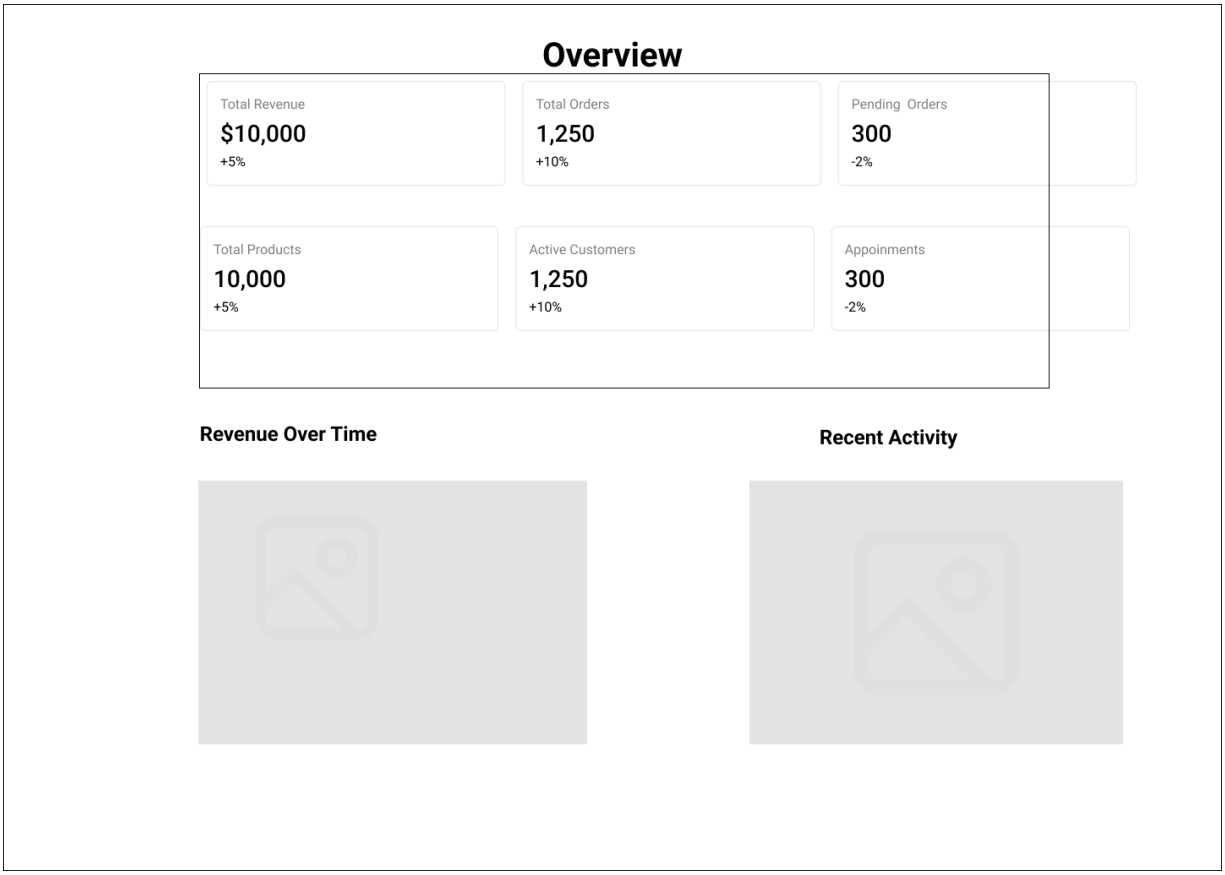



*Figure 5.8 Wireframe of Admin Overview Page*

Admin Manage Orders

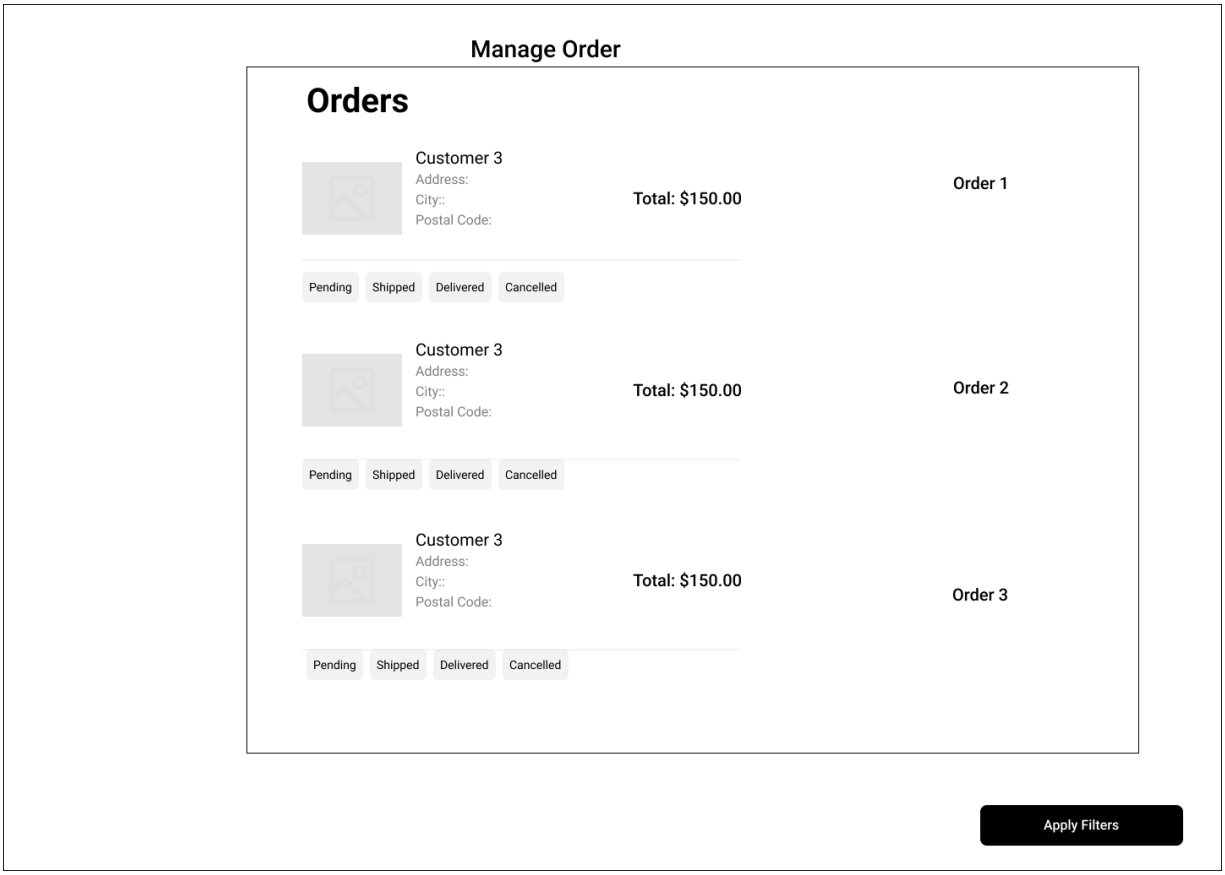



*Figure 5.9 - Wireframe of Admin Mange Orders*

Admin Manage Appointments

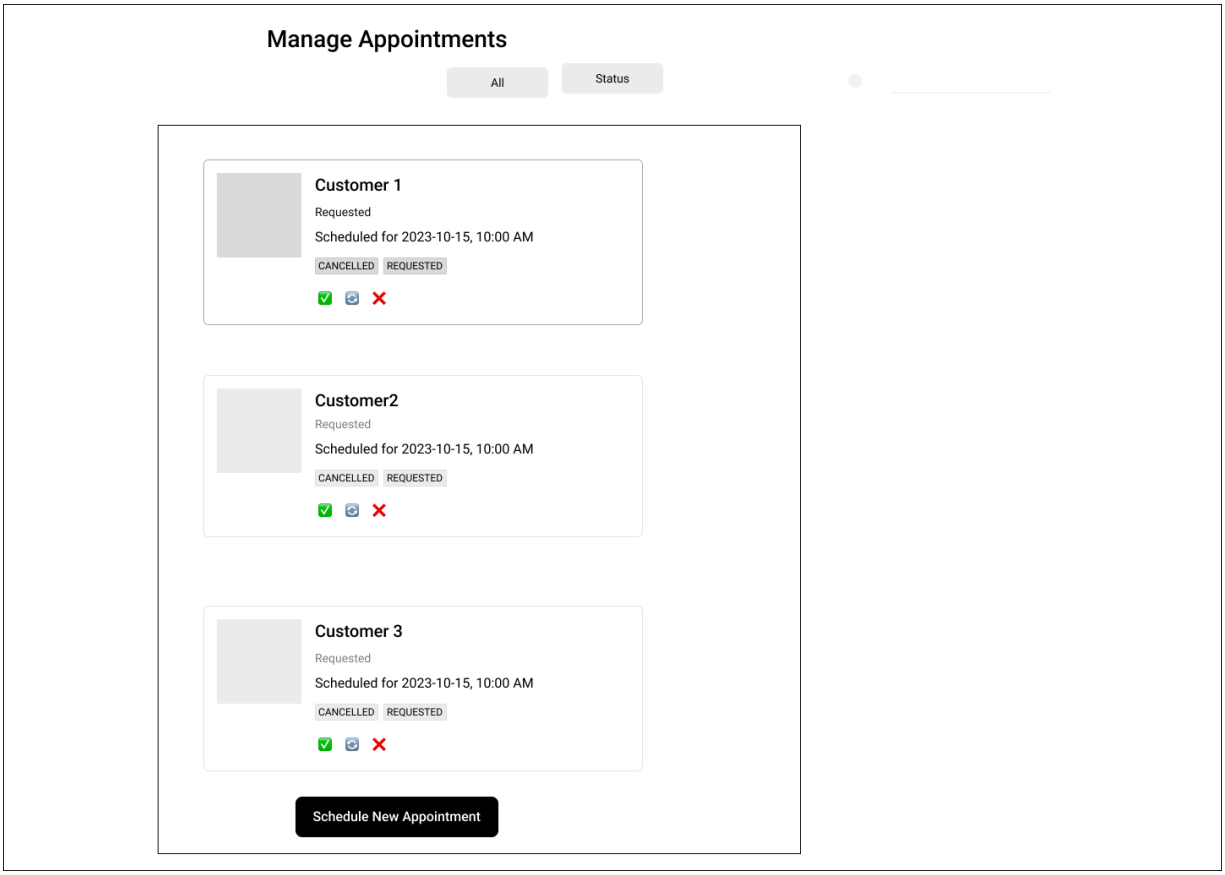



*Figure 5.10 Wireframe of Admin Manage Appointment*

Admin Manage Users

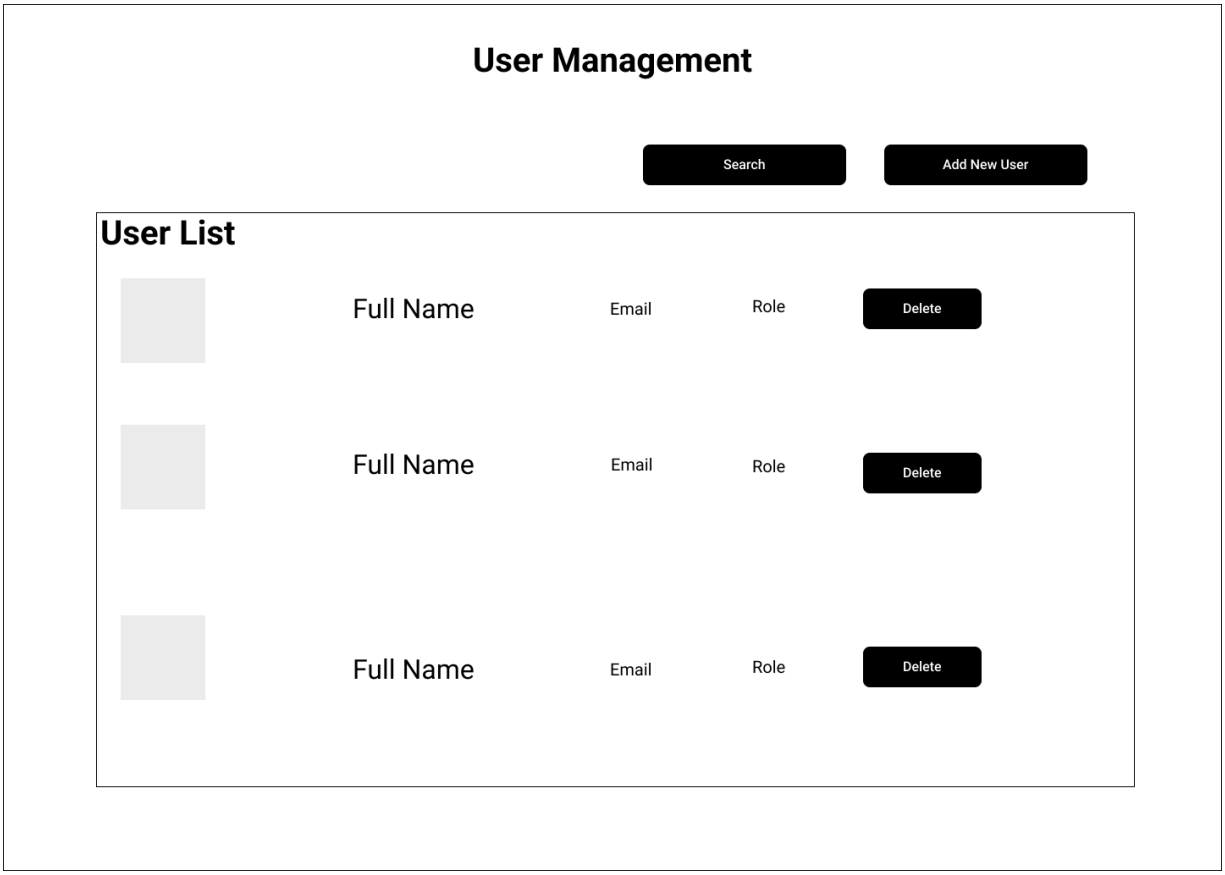



*Figure 5.11 Wireframe of Admin Manage Users*

## 5.2 User flow / Navigation Diagrams

### 5.2.1 Customer Flow

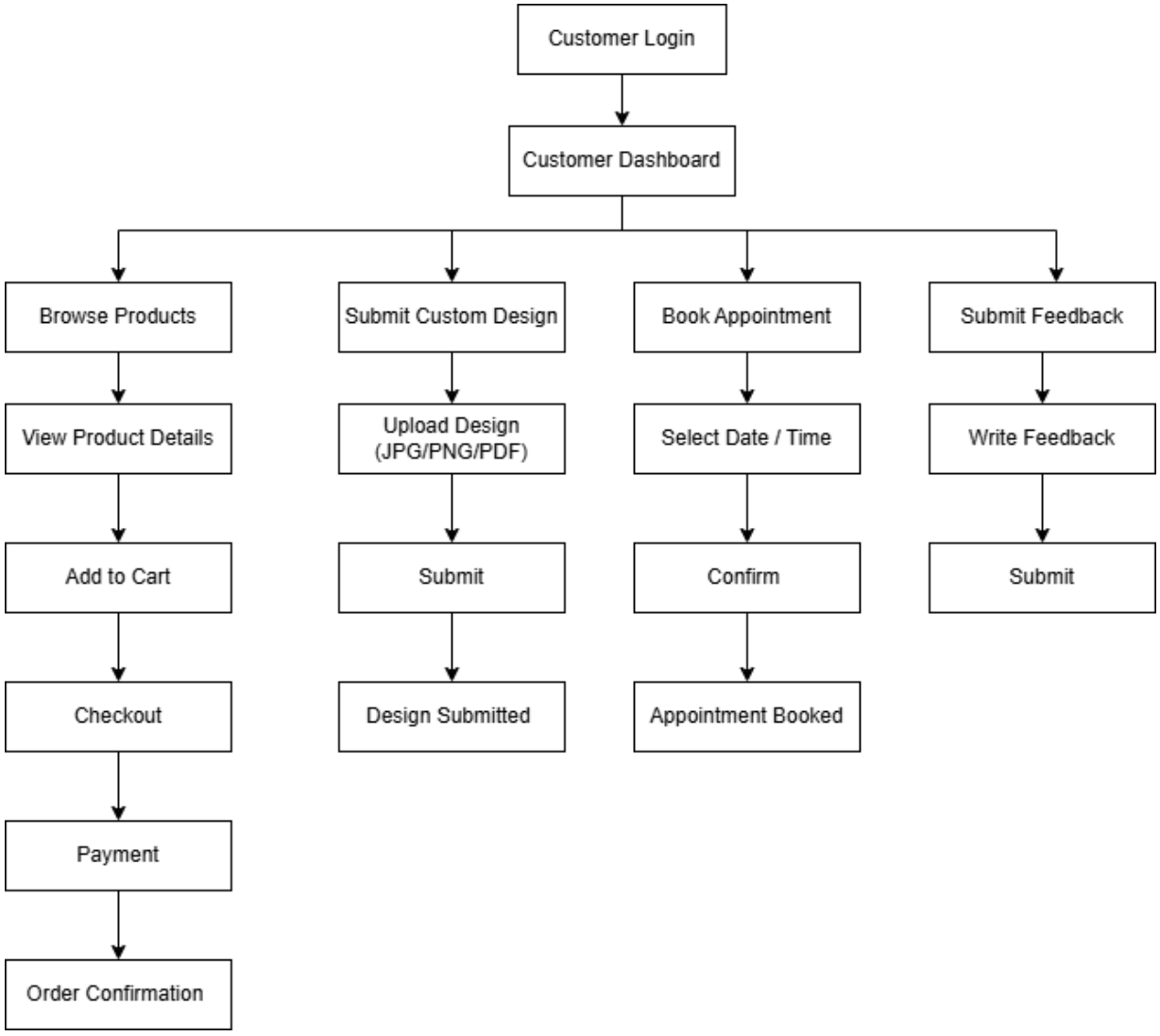



*Figure 5.12 User Flow Diagram of Customer*

5.2.2 Admin Flow

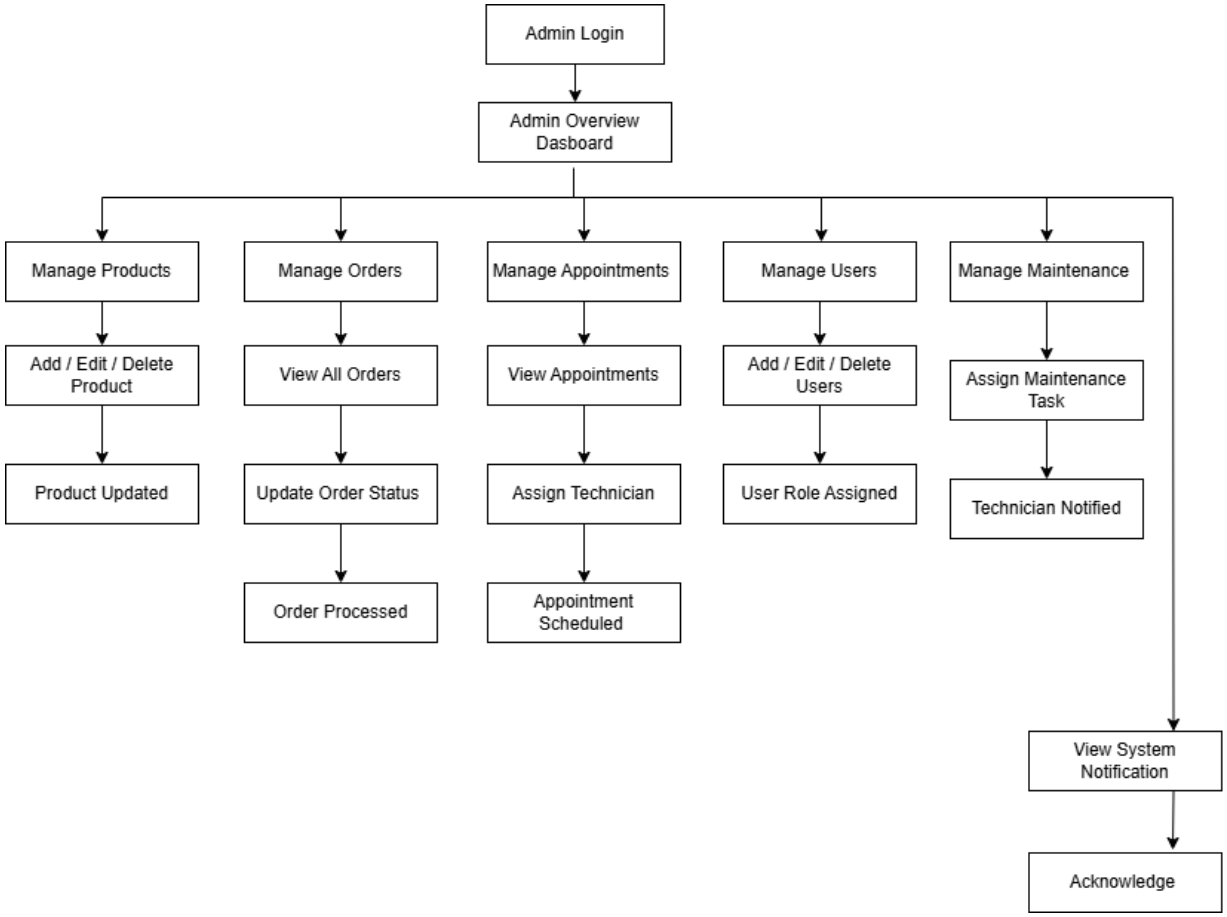



*Figure 5.13 User Flow Diagram of Admin*

5.2.3 Technician Flow

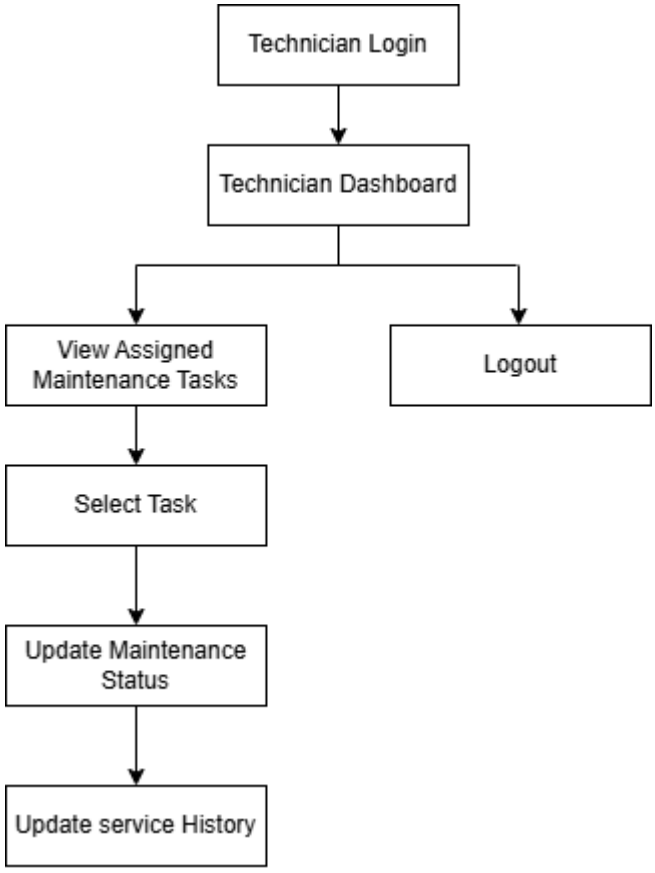



*Figure 5.14 User Flow Diagram of Technician*

**5.3 Accessibility Considerations (WCAG 2.1 Compliance)**

The system is designed to ensure accessibility for a wide range of users, including individuals with disabilities, in accordance with the Web Content Accessibility Guidelines (WCAG) 2.1.

The following accessibility features are incorporated

- Perceivable Content

  All images and visual elements include alternative text (alt text) to support screen readers. Sufficient color contrast is maintained between text and background to improve readability for visually impaired users.
- Keyboard Accessibility

  All interactive elements such as forms, buttons, and navigation menus are fully accessible via keyboard navigation without requiring a mouse.
- Readable and Understandable Content

  Clear and simple language is used throughout the interface. Form inputs include labels and error messages to assist users in understanding required actions.
- Consistent Navigation

  The system maintains consistent layout and navigation patterns across all pages, enabling users to easily predict and understand how to interact with the system.
- Assistive Technology Support

  The application is compatible with assistive technologies such as screen readers by using semantic HTML elements and ARIA (Accessible Rich Internet Applications) attributes where necessary.
- Responsive Design

  The user interface is responsive and adapts to different screen sizes and devices, ensuring accessibility across desktops, tablets, and mobile devices.
- Error Identification and Feedback

  Input validation errors are clearly indicated with descriptive messages, helping users to correct mistakes easily.

The system aims to meet WCAG 2.1 Level AA standards, ensuring an inclusive and user-friendly experience for all users.

# CHAPTER 6 SECURITY DESIGN

## 6.1 Authentication Mechanism

A JSON Web Token (JWT)-based authentication mechanism is implemented for all user sessions in the GMC Lanka system.

The authentication process is carried out as follows:

- The user's email and password are submitted via HTTPS to the POST /auth/login API endpoint.
- The user record is retrieved from the MySQL database, and the submitted password is compared with the stored hash using the BCryptPasswordEncoder provided by the Spring Security framework.
- Upon successful verification, a signed JWT containing the user ID, role (Admin, Customer, or Technician), and an expiry time of 8 hours is generated.
- The generated JWT is returned to the client and is stored in an httpOnly, Secure, SameSite=Strict cookie to prevent access via XSS attacks.
- For every subsequent API request, the JWT is included in the Authorization: Bearer <token> header.
- The token signature and expiry are validated on each protected route.
- Session tokens are automatically invalidated after 8 hours, and re-authentication is required.
- For administrative users, Multi-Factor Authentication (MFA) is enforced using a Time-Based One-Time Password (TOTP) application (e.g., Google Authenticator).

## 6.2 Authorization Levels

Role-Based Access Control (RBAC) is enforced across all API endpoints. Access permissions are determined based on the role of claim extracted from the validated JWT.

The authorization levels are defined as follows

- Admin (Full Access)
  All system functions, including user management, product management, order processing, billing, maintenance scheduling, and system noSequelize tifications, are permitted.
- Customer (Restricted Access)
  Catalogue browsing, order placement, order tracking, custom design submission, appointment booking, payment processing, and feedback submission are permitted.
- Technician (Limited Access)
  Assigned maintenance tasks can be viewed, and maintenance status and service history can be updated.
- Unauthenticated Users (Public Access)
  Only the public product listing page can be accessed; all other routes are redirected to the login page.

Any unauthorized access attempt (e.g., a Customer accessing Admin-only endpoints) is rejected with a 403 Forbidden response.

### 6.3 Data Protection Approach

Sensitive user data is protected using multiple security mechanisms.

- Personally identifiable information (PII), including names, email addresses, phone numbers, and order history, is stored in Amazon RDS within a private VPC subnet that is not accessible from the public internet.
- Passwords are never stored in plain text. Instead, they are securely hashed using the bcrypt algorithm with an appropriate cost factor.
- Payment card data is not stored on GMC Lanka servers. All payment processing is handled by Stripe's PCI-DSS Level 1 compliant infrastructure using Stripe Elements.
- Customer data is processed in compliance with the Sri Lanka Personal Data Protection Act (PDPA) 2022.

- Database access is restricted exclusively to the EC2 application server through AWS Security Group rules, and no direct public access is permitted.

### 6.4 Input Validation And Sanitization Strategy

Various security threats are mitigated through appropriate validation and sanitisation techniques.

- SQL Injection
  SQL injection attacks are prevented through the use of parameterised queries implemented via JPA (Hibernate). The use of raw SQL queries is avoided.
- Cross-Site Scripting (XSS)
  XSS attacks are mitigated by encoding all user-generated content before rendering and enforcing Content Security Policy (CSP) headers.
- Cross-Site Request Forgery (CSRF)
  CSRF attacks are prevented using SameSite=Strict cookies and CSRF tokens for all state-changing requests.
- Invalid Input Handling
  Input validation is enforced through server-side validation using the Spring Validation framework (Jakarta Validation / Hibernate Validator). Client-side validation is used only to enhance user experience.
- File Upload Attacks
  File upload vulnerabilities are prevented by validating MIME types (JPG, PNG, PDF) and restricting file sizes to a maximum of 10MB.
- Brute Force Attacks
  Brute force attacks are mitigated using rate limiting (e.g., a maximum of 5 login attempts per 15 minutes per IP address) and temporary account lockout after repeated failed login attempts.

### 6.5 Encryption Standards For Data at Rest and in Transit

Appropriate encryption standards are applied to protect data both in transit and at rest.

- Data in Transit
  TLS 1.2 and TLS 1.3 protocols are enforced for all communications via HTTPS. Older versions (TLS 1.0 and 1.1) are disabled.
- Data at Rest (Database)
  Encryption using AES-256 is enabled in Amazon RDS through AWS Key Management Service (KMS).
- Data at Rest (Storage)
  Amazon S3 server-side encryption (SSE-S3) using AES-256 is applied to all stored files and images.
- Passwords
  Passwords are secured using bcrypt hashing with salting, ensuring a one-way encryption mechanism.
- JWT Tokens
  JWTs are signed using the HMAC SHA-256 algorithm with a secure server-side secret key.
- Payment Data
  Payment information is encrypted end-to-end by Stripe and is never transmitted to or stored on GMC Lanka servers.

# CHAPTER 7 DEPLOYMENT DESIGN

## 7.1 Hosting Plan

The system is deployed using cloud infrastructure provided by Amazon Web Services (AWS) to ensure scalability, reliability, and high availability.

The frontend application is hosted using AWS S3 (Simple Storage Service) as a static website and is delivered through AWS CloudFront, which acts as a Content Delivery Network (CDN) to improve performance and reduce latency for end users.

The backend application, developed using Spring Boot, is deployed on AWS EC2 (Elastic Compute Cloud). The application runs as a standalone executable (.jar file) within a virtual server environment, allowing full control over server configuration and resource management.

The database layer is implemented using AWS RDS (Relational Database Service), which provides a managed database solution supporting MySQL or PostgreSQL. Automated backups, scaling capabilities, and high availability features are utilized to ensure data reliability.

Static assets such as images and uploaded files are stored in AWS S3, ensuring secure and scalable storage.

## 7.2 Environment Specification

The system is deployed across multiple environments to support the software development lifecycle:

- Development Environment
  Used for implementing new features and performing initial testing.
- Staging Environment
  Used for system testing and validation before deployment to production.

- Production Environment
  The live system is accessible to end users, configured for high availability and performance.

### 7.3 Deployment Strategy

The source code is managed using version control systems such as GitHub.

A Continuous Integration and Continuous Deployment (CI/CD) pipeline may be utilized using tools such as AWS CodePipeline or GitHub Actions. This enables automated building, testing, and deployment of the application to the AWS environment.

### 7.4 Network and Security Configuration

The system is deployed within an AWS Virtual Private Cloud (VPC) to ensure network isolation and security.

Security Groups are configured to control inbound and outbound traffic, allowing only necessary ports such as HTTP (80) and HTTPS (443). The database is deployed in a private subnet and is not directly accessible from the public internet.

Identity and Access Management (IAM) roles are used to securely manage access to AWS resources.

### 7.5 Network Diagram

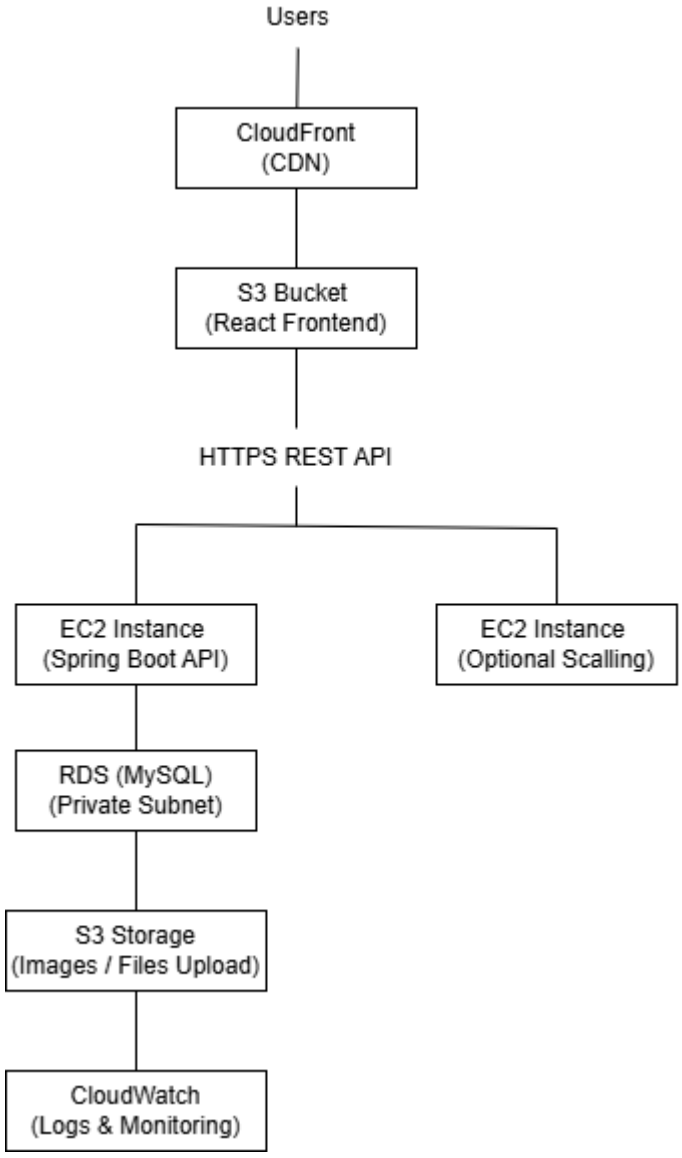



*Figure 7.1 Network Diagram*

## CHAPTER 8 REFERENCES


[1] IEEE, "IEEE Recommended Practice for Software Requirements Specifications," IEEE Std 830-1998, New York, NY, USA: IEEE, 1998.

[2] ISO/IEC, "ISO/IEC 25010:2011: Systems and Software Engineering — Systems and Software Quality Models," Geneva, Switzerland: ISO/IEC, 2011.

[3] Stripe Inc., "Stripe API Reference," Stripe Documentation. [Online]. Available: https://stripe.com/docs/api. [Accessed: Mar. 21, 2026].

[4] Amazon Web Services, "AWS Documentation," Amazon Web Services. [Online]. Available: https://docs.aws.amazon.com. [Accessed: Mar. 21, 2026].

[5] Oracle Corporation, "MySQL 8.0 Reference Manual," Oracle. [Online]. Available: https://dev.mysql.com/doc. [Accessed: Mar. 21, 2026].

[6] React, "React Documentation," Meta Open Source. [Online]. Available: https://react.dev. [Accessed: Mar. 21, 2026].

[7] OWASP Foundation, "OWASP Top Ten 2021," [Online]. Available: https://owasp.org/Top10. [Accessed: Mar. 21, 2026].

[8] W3C, "Web Content Accessibility Guidelines (WCAG) 2.1," [Online]. Available: https://www.w3.org/TR/WCAG21. [Accessed: Mar. 21, 2026].

[9] JSON Web Tokens, "Introduction to JSON Web Tokens," [Online]. Available: https://jwt.io/introduction. [Accessed: Mar. 21, 2026].

[10] Software Engineering Teaching Unit, "SENG 31242 System Design Specification Guidelines," Faculty of Science, University of Kelaniya, Sri Lanka, 2026.